Sprint 1 Final Submission Document

1. Sprint Details

- **Sprint Number: 2**
- **Sprint Pod Name:** EduSphere – E-Learning Management System
- **Pod Members:**
  - Member 1 – Aniket Das
  - Member 2 – Anish Dey
  - Member 3 – Arya Mane
  - Member 4 – Aymaan Shabbir
  - Member 5 – Bidisha Talukdar
  - Member 6 – Disha Makal
- **Submission Date:** 19-06-2025

2. Sprint Goal

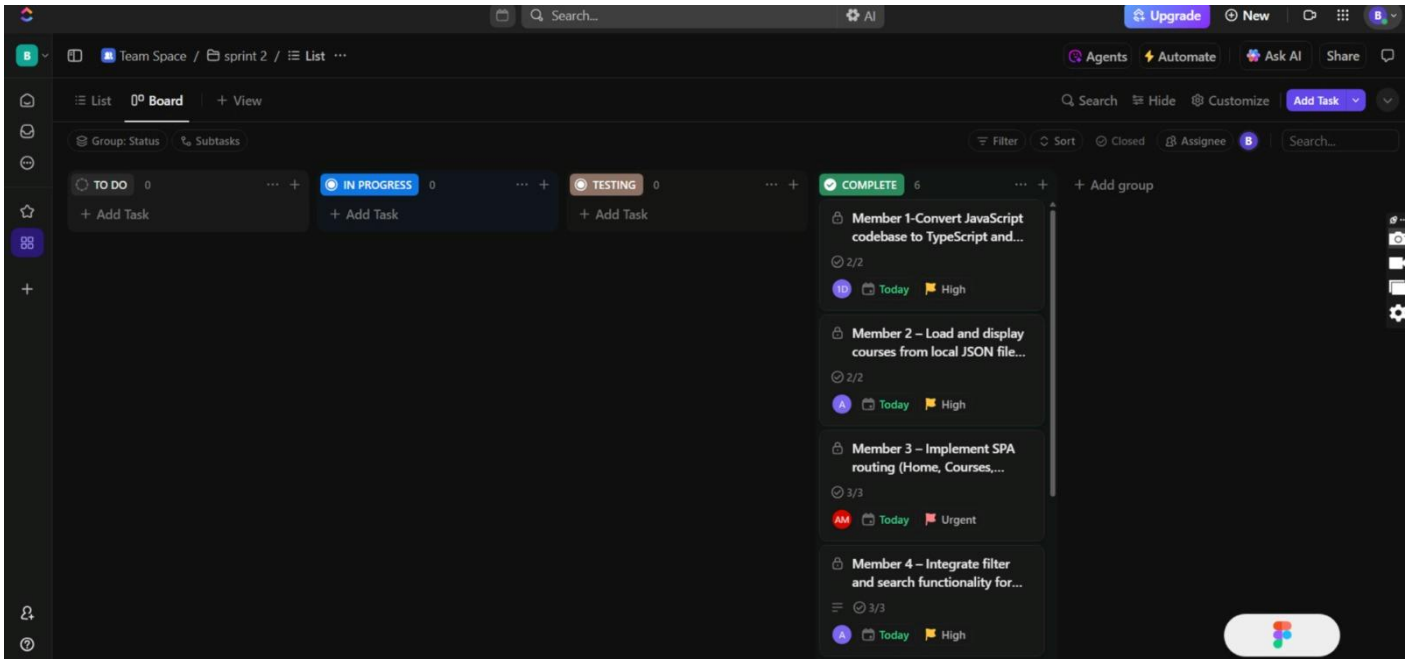
Implement single-page routing, dynamic course rendering and PWA setup to enhance interactivity and offline capabilities. The goal is to transition the project into a scalable, component-driven architecture using TypeScript, enabling modular development and better code maintainability. By integrating lazy loading and search or filter functions, we aim to optimize performance and user experience. Additionally, the implementation of PWA features ensures offline accessibility and installability, aligning the application with modern web standards.

3. User Stories Completion

User Story ID	Description	Assigned To	Status	Acceptance Criteria Met
US1	Converted JavaScript codebase to TypeScript and defined models for course and User	Member 1	Done	Yes
US2	Loaded and displayed courses from local JSON file using Fetch API	Member 2	Done	Yes
US3	Implemented SPA routing (Home, Courses, Dashboard, Contact)	Member 3	Done	Yes
US4	Integrated filter and search functionality for course listings	Member 4	Done	Yes

US5	Implemented lazy loading for course detail and dashboard pages	Member 5	Done	Yes

US6	Enabled installable PWA features (manifest, service worker)	Member 6	Done	Yes



4. Code Submission

GitHub Repository Link: https://github.com/DISHA-6/E_Learning_System

Included Features:

- SPA routing using a client-side router (Home, Courses, Dashboard pages)
- Data loading from local JSON via Fetch API
- Dynamic rendering of course cards on the Courses page
- Filter and search functionality integrated for course listings
- Lazy loading implemented for Course Detail and Dashboard routes
- Progressive Web App (PWA) features:
    - Web App Manifest
    - Service Worker for offline caching
    - Install prompt enable

5. Sprint Review Document

What Was Planned:

In Sprint 2, the focus shifted from static UI to enabling interactivity and performance through dynamic content rendering and offline capabilities. The key objectives included:

- Implementing Single Page Application (SPA) routing between core sections (Home, Courses, Dashboard)
- Loading course data dynamically from a local JSON file using the Fetch API
- Adding search and filter functionality for course listings
- Refactoring code to TypeScript and creating models/interfaces for structure and scalability
- Introducing lazy loading for components like Course Details and Dashboard to optimize performance
- Setting up Progressive Web App (PWA) features like manifest and service worker for offline access and installability

What Was Delivered:

All planned features were successfully developed and tested across different user roles. The completed deliverables include:

- SPA Routing:
  - Smooth navigation across Home, Courses, Dashboard, and Contact pages without reloading
- Dynamic Course Rendering:
  - Courses fetched from a local JSON file and displayed on the Courses page
- Search and Filter Functionality:
  - Users can search courses by title and filter them by category or availability
- TypeScript Refactoring:
  - Defined interfaces for Course and User
  - Converted key modules from JavaScript to TypeScript for better type safety
- Lazy Loading:
  - Implemented route-based code splitting for improved performance on navigation

- Progressive Web App Setup:
    - Added manifest.json for app installability
    - Registered service-worker.js to cache essential resources for offline use

Key Limitations:

- Offline mode does not include custom fallback pages for failed network requests
- Filter options are currently limited to single-criteria filtering
- Cache versioning and update strategy for service worker is basic
- No loading indicators for lazy-loaded routes
- Data source is still local; backend/API integration is pending

Challenges Faced:

1. Role-Based Navigation Complexity

- Implementing conditional rendering purely in JavaScript required precise logic and DOM manipulation.
- Since no backend was available, we had to simulate login flows manually for different user roles (Admin/User), which added complexity to the routing and visibility logic.

2. Course Add Persistence

- Admin-added courses could not be reflected in the User view due to the absence of dynamic data storage or real-time backend integration.
- Course data remained static and hard-coded in JSON, limiting interaction and real-time updates.

3. Modal UI Interference

- While integrating video preview modals with the existing Bootstrap layout, layout-breaking issues were observed, especially on smaller screen devices.
- Modals sometimes overlapped or pushed content unexpectedly, affecting responsiveness.

4. Team Coordination

- Initial Git merge conflicts occurred due to inconsistent file naming conventions and overlapping changes in shared files like index.html and router.js.
- Required multiple sync-up calls and code reviews to maintain consistency and resolve conflicts.

Key Learnings:

- Improved understanding of TypeScript interfaces and how they support scalable code
- Learned practical usage of Fetch API to consume JSON data
- Hands-on experience in SPA routing and route-based code splitting
- Understood how service workers operate, cache strategies, and PWA manifest configurations
- Exposure to real-world debugging and browser DevTools (for PWA and routing issues)

What Went Well:

- Effective task distribution among members allowed parallel development
- Members helped each other troubleshoot TypeScript and routing issues
- All core stories completed and tested successfully within the sprint timeline
- Regular sync-ups helped maintain project flow and resolve blockers early

What Didn't Go Well:

- Delayed progress during the TypeScript migration phase due to unfamiliarity
- Missed early testing, leading to a last-minute rush for resolving PWA cache issues
- Lack of reusable loading/skeleton UI impacted visual feedback during lazy loading

Action Items for Next Sprint:

- Begin early testing with dummy API integrations and edge case scenarios
- Improve documentation for components and data flow
- Add error boundary components for better runtime handling
- Explore integration of real-time backend (e.g., Firebase or mock REST API)
- Enhance UI/UX with animations, loaders, and better feedback mechanisms